

02 배치 개발 가이드

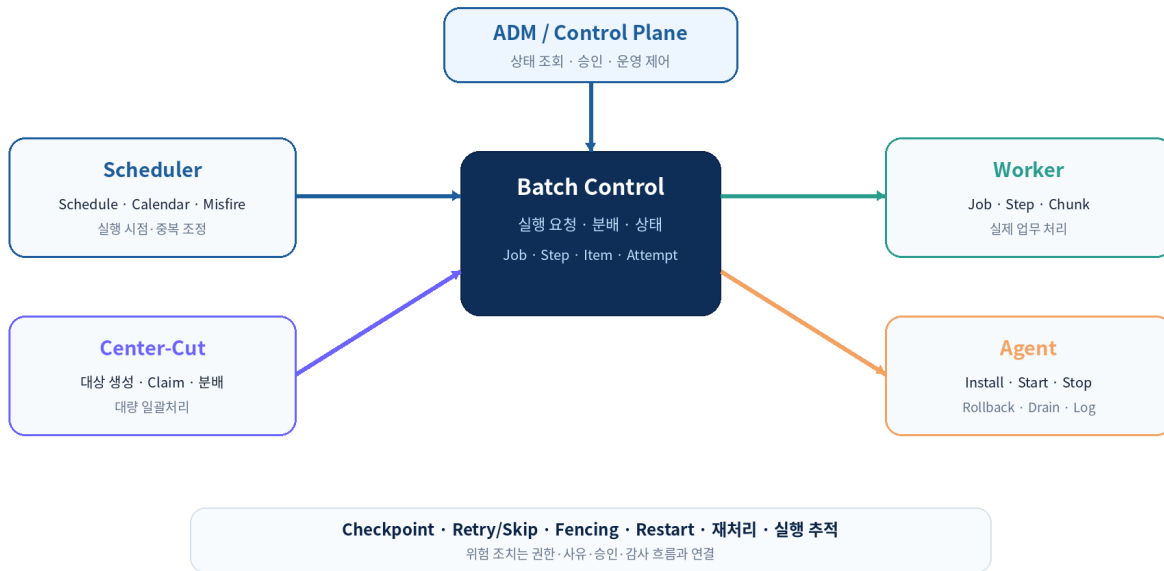
Job·Step 설계부터 Scheduler·Worker·Agent·Center-Cut 실행과 재처리까지 이어지는 Batch 실무 매뉴얼

기준 Source: master bd5fcd7707cfbba366071e7e4556c5cbb9b974bc · 2026-08-15

1. Batch Runtime 구성

CPF Batch Runtime

스케줄, 분산 실행, Node 제어, 대량 일괄처리를 하나의 실행·운영 모델로 구성



Scheduler·Worker·Agent·Center-Cut 과 운영 제어

구성	주요 역할	사용 기준
Generated Batch	업무 Job·Step 구현	업무 Batch 코드
Scheduler	Schedule·Calendar·Misfire·중복 실행 조정	정기·예약 실행
Worker	Job·Step·Chunk 실제 처리	단일/분산 실행
Agent	Artifact 설치·Start/Stop·Rollback·Drain	Node 실행 제어
Center-Cut	대상 생성·Claim·분배·집계	대량 일괄처리
운영 제어	실행·상태·실행 시도·운영 명령	통합 운영

2. Job·Step 공개 Annotation

Annotation	옵션	의미	적용 기준
`@CpfBatchJob`	`value`	안정된 Job ID	운영/재시작 기준 ID
`@CpfBatchJob`	`restartable`	재시작 허용	기본 true
`@CpfBatchJob`	`maxConcurrentExecutions`	동시 실행 상한	기본 1

Annotation	옵션	의미	적용 기준
<code>@CpfBatchStep`</code>	<code>`value`</code>	안정된 Step ID	Job 내부 식별
<code>@CpfBatchStep`</code>	<code>`order`</code>	Step 순서	명시 순서가 필요할 때
<code>@CpfBatchStep`</code>	<code>`idempotent`</code>	재실행 안전성	기본 true

```

@CpfBatchJob(value = "MBR_DAILY_MEMBER", restartable = true, maxConcurrentExecutions = 1)
public class MemberDailyJob {
    @CpfBatchStep(value = "LOAD_MEMBER", order = 10, idempotent = true)
    public void loadMember() { ... }
}

```

3. 실행 요청 선택

필요한 작업	공개 API	특징	주의
즉시 실행	<code>`CpfBatchExecutionRequest.run()`</code>	새 실행 요청	동일 실행 방지 Key 검토
스케줄 실행	<code>`scheduledRun()`</code>	scheduleId 보존	Scheduler 에서 사용
실패 실행 재시작	<code>`restart()`</code>	같은 JobInstance 에서 이어서 실행	restartable 정책 확인
새 Instance 재수행	<code>`rerun()`</code>	기존 파라미터 기반 신규 실행	중복 Side Effect 검토
재시도	<code>`retry()`</code>	실패 실행을 기준으로 요청	재시도 가능 상태 확인
중지	<code>`stop()`</code>	실행 중지 요청	중지 가능 지점/Checkpoint 확인
온라인 접수	<code>`onDemand()`</code>	standardBatchId · businessDate · idempotencyKey 보존	API 접수형 Batch

4. Job·Step 설계 기준

- Job ID · 업무일자 · 업무 Key 를 먼저 정하고 동일 실행 판정 기준을 정의합니다.
- Step 은 재시작 가능한 업무 단위와 Transaction 경계에 맞춰 나눕니다.
- Chunk 크기는 처리량만이 아니라 Lock 시간 · Rollback 범위 · 재처리 비용을 함께 고려합니다.
- 외부 Side Effect 가 있으면 멍등키와 결과 확인 방식을 설계합니다.
- 대량 실행은 Worker 병렬화 전에 대상 분할 · 소유권 · Fencing 기준을 먼저 정합니다.

5. Scheduler·Worker

항목	Scheduler	Worker
책임	실행 시점 · 중복 조정 · 의존 관계	업무 Job/Step 실제 실행
확장	Leader/Fencing · 다중 Scheduler	Worker 수평 확장
장애 기준	Misfire · 중복 요청 · Leader 변경	Heartbeat · 실행 시도 · 중단/재시작
운영 식별자	scheduleId · executionId	workerId · serverInstanceId · executionId

6. Center-Cut

단계	처리	운영 확인
대상 생성	처리 대상과 실행 기준 고정	총 대상 수 · 기준 시점
분할/Claim	Worker 가 처리 범위를 소유	중복 Claim · 작업 소유권
실행	Item/Chunk 처리	실행 시도 · 상태 · 오류
Fencing	오래된 Worker 결과 차단	소유권 Token
집계	성공/실패/미확정 구분	완료 수 · 실패 수 · UNKNOWN
재처리	실패/미처리만 재선정	원 실행과 재처리 연결

7. Agent 운영 명령

명령	역할	안전 기준
Install	검증된 Artifact 설치	checksum/version 확인
Start	Runtime 시작	중복 기동 · Port 확인
Stop	정상 중지	진행 작업 처리 정책 확인
Restart	중지 후 재기동	상태 · 승인 확인
Rollback	이전 Artifact 복귀	대상 Version 고정
Drain / Resume	신규 작업 유입 중지/재개	기존 실행과 구분
Log 조회	제한된 운영 로그 수집	민감정보 마스킹

8. 실행 상태와 재처리 판단

상태/상황	판단	후속 처리
정상 완료	결과 확정	종료
업무 실패	업무 조건 실패 확정	입력/업무 규칙 수정 후 재실행 판단
기술 실패	기술 오류 확정	원인 제거 후 restart/retry
UNKNOWN	결과 확정 불가	Blind Retry 금지, Ledger/실제 결과 확인
Worker 단절	Heartbeat 소실	작업 소유권/Fencing 확인 후 재판정
부분 성공	완료 · 실패 대상 혼재	정상 완료 제외 후 재처리

9. 운영 조회 API

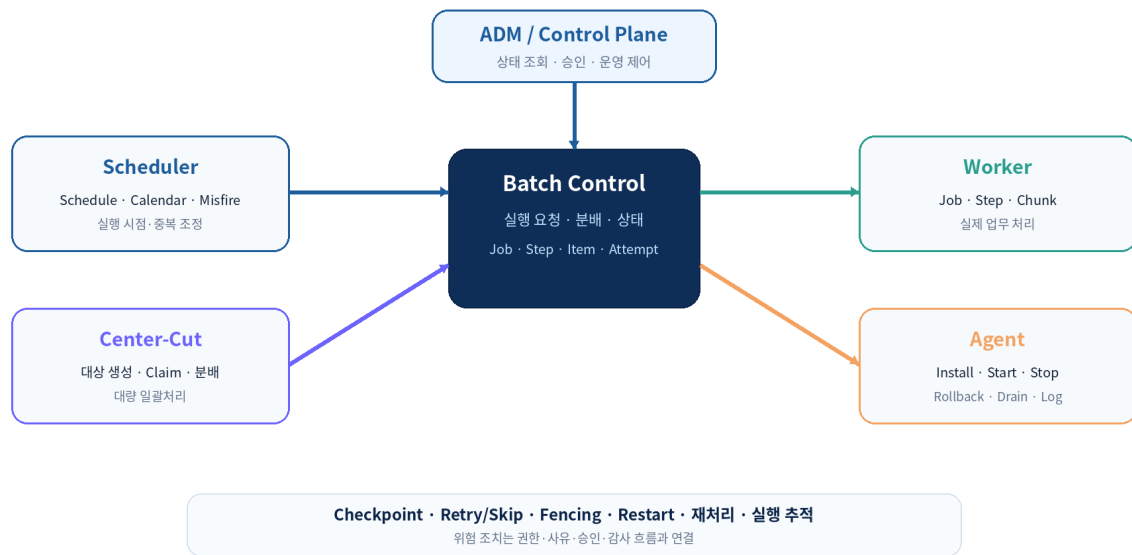
API	주요 조회	사용 위치
<code>`CpfBatchOperationsPort.findJobs()`</code>	Job 목록	ADM/운영 조회
<code>`findSchedules()`</code>	Schedule 목록	Scheduler 운영
<code>`findExecutions(...)`</code>	Job · transactionId · worker · instance 별 실행	실행 추적

API	주요 조회	사용 위치
`findExecutionDetail()`	단일 실행 상세	원인 분석
`findWorkers()`	Worker 상태	분산 실행 확인
`findInfrastructureSnapshot()`	Instance·Worker·Target 상태	인프라 확인
`findRecoverySnapshot()`	Ghost·Lock·운영 명령	재처리/복구 판단

참조 공개 API `cpf-batch/api/\*\*` · Runtime `cpf-batch/{scheduler,worker,agent,center-cut,control-plane}/\*\*` · 예제 `cpf-education/\*\*`

CPF Batch Runtime

스케줄, 분산 실행, Node 제어, 대량 일괄처리를 하나의 실행·운영 모델로 구성



Scheduler·Worker·Agent·Center-Cut 의 실행·운영 관계